

[DHS/2024/Q1]

1 You started a new music streaming platform called Spotify. In order to ensure a smooth and seamless experience for your users, your server compresses the music data to reduce its size before transmitting the compressed data to the user's app. The user's app then decompresses the received data back into its original form to play the music.

- **Step 1:** Server reads the data of the audio file. [Note: Each byte is represented by 2 hexadecimal digits and is stored as an element in the list.]

```
['61', '61', '62', '62', '62', '62', '62', '62', '63', '64', '85',  
'6F', '2A', '6F', '65', '65', '65', '65', '65', '65', '65',  
'65', '65', '6A', '6B', '72', '74', '74', '74', '83', '85',  
'86', '87', '88', '83', '85', '86', '87', '88', '83', '85',  
'86', '87', '88', '92', '95', '9B', '9E', '9E', '9E', '9E',  
'9E', '9E', '9E', '9D', '9E', '9E', '9E', '9E', '9E', '63',  
'64', '85', '6F', '2A', '6F', '63', '64', '85', '6F', '2A',  
'6F']
```

- **Step 2:** Server searches for the given bytes pattern and replaces it with a single byte representation. E.g. replacing the bytes pattern 636485 to C0, 6F2A6F to C1 and 8385868788 to C2, resulting in the following compressed data:

```
['61', '61', '62', '62', '62', '62', '62', '62', 'C0', 'C1', '65',  
'65', '65', '65', '65', '65', '65', '65', '65', '6A', '6B',  
'72', '74', '74', '74', 'C2', 'C2', 'C2', '92', '95', '9B',  
'9E', '9E', '9E', '9E', '9E', '9E', '9E', '9D', '9E', '9E',  
'9E', '9E', '9E', 'C0', 'C1', 'C0', 'C1']
```

- **Step 3:** Server to send the compressed data and the key in string format to the user's app (client). [Note: the key stores all the bytes pattern and its byte representation applied to the original data.]

Compressed Data sent to user:

```
61616262626262C0C16565656565656565656A6B72747474C2C2C292959B  
9E9E9E9E9E9E9E9D9E9E9E9E9EC0C1C0C1
```

Key sent to user:

```
C0,636485,C1,6F2A6F,C2,8385868788
```

- **Step 4:** The user's app (client) decompresses the data received from the server with the key provided back to its original form.

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [ ]: #Task 1.1  
        Program code
```

Output:

Task 1.1

The text file `Audio.txt` stores the bytes for a song. Each byte is represented by 2 hexadecimal digits and each byte is written on a new line in the text file. 61 is the first byte in the song.

Write the function `readfile(filename)` that takes a file name as a parameter, reads in the data and returns the data as a list of bytes. [2]

Note: for **Task 1.2** and **Task 1.3**, you are **not allowed** to use any built-in functions or any libraries/modules to **match and replace** the `bytes_pattern` and `byte_rep`.

[DHS/2024/Q1 Cont']

Task 1.2

Write the function `compress (bytes_list, bytes_pattern, byte_rep)` that takes in a list of bytes, searches for the bytes pattern in the list and replace the affected bytes with the byte representation. The function should return the compressed list of bytes. [6]

Call your `readfile` function with `Audio.txt` as the parameter and test your `compress` function with the returned list of bytes together with the bytes pattern `636485` and byte representation `C0`. [1]

Task 1.3

Write the function `decompress (compressed_list, byte_rep, bytes_pattern)` that takes in the compressed list of bytes, searches for the byte representation in the list and replace the byte representation with the bytes pattern. The function should return the decompressed list of bytes. [4]

Test your `decompress` function with the compressed list of bytes from Task 1.2 together with the byte representation `C0` and bytes pattern `636485`. [1]

Task 1.4

Using socket programming, complete both the server program used by Spotify to send the compressed music data & key, and the client program used by the users to receive the compressed music data & key and decompress the data back to its original form. [9]

- Copy your functions `readfile(filename)` and `compress (bytes_list, bytes_pattern, byte_rep)` into the server program.
- Copy your function `decompress (compressed_list, byte_rep, bytes_pattern)` into the client program.
- Complete the client program to:
 - receive the compressed data and key from the server program.
 - decompress the compressed data received.
 - display the decompressed data as a list of bytes.

You are provided with the server and client template programs, `SERVER.py` and `CLIENT.py` respectively. Complete both programs and rename as

```
CLIENT_<your name>_<centre number>_<index number>.py
SERVER_<your name>_<centre number>_<index number>.py
```

Study the sample program output in the next two pages to determine your code design, output format and socket protocol. User inputs are underlined.

Sample client program:

```
-----
CLIENT OPEN
-----
```

```
Raw compressed data received:
```

```
61616262626262C0C16565656565656565656A6B72747474C2C2C292959B9E9
E9E9E9E9E9E9D9E9E9E9E9EC0C1C0C1
```

```
Key received: C0,636485,C1,6F2A6F,C2,8385868788
```

```
Decompressed data: ['61', '61', '62', '62', '62', '62', '62',
'63', '64', '85', '6F', '2A', '6F', '65', '65', '65', '65', '65',
'65', '65', '65', '65', '6A', '6B', '72', '74', '74', '74', '83',
'85', '86', '87', '88', '83', '85', '86', '87', '88', '83', '85',
'86', '87', '88', '92', '95', '9B', '9E', '9E', '9E', '9E', '9E',
'9E', '9E', '9D', '9E', '9E', '9E', '9E', '9E', '63', '64', '85',
'6F', '2A', '6F', '63', '64', '85', '6F', '2A', '6F']
```

```
-----
CLIENT CLOSED
-----
```

[DHS/2024/Q1 Cont']

```
Sample server program:
-----
SERVER OPEN
-----

Waiting for client request
-----
original data: ['61', '61', '62', '62', '62', '62', '62', '63',
'64', '85', '6F', '2A', '6F', '65', '65', '65', '65', '65', '65',
'65', '65', '65', '6A', '6B', '72', '74', '74', '74', '83', '85',
'86', '87', '88', '83', '85', '86', '87', '88', '83', '85', '86',
'87', '88', '92', '95', '9B', '9E', '9E', '9E', '9E', '9E', '9E',
'9E', '9D', '9E', '9E', '9E', '9E', '9E', '63', '64', '85', '6F',
'2A', '6F', '63', '64', '85', '6F', '2A', '6F']

Enter the bytes pattern to be replaced: 636485
Enter the value to replace the above bytes pattern: C0
Do you want to continue compressing? [Y/N]: Y

Enter the bytes pattern to be replaced: 6F2A6F
Enter the value to replace the above bytes pattern: C1
Do you want to continue compressing? [Y/N]: Y

Enter the bytes pattern to be replaced: 8385868788
Enter the value to replace the above bytes pattern: C2
Do you want to continue compressing? [Y/N]: N

compressed_data: ['61', '61', '62', '62', '62', '62', '62', 'C0',
'C1', '65', '65', '65', '65', '65', '65', '65', '65', '65', '6A',
'6B', '72', '74', '74', '74', 'C2', 'C2', 'C2', '92', '95', '9B',
'9E', '9E', '9E', '9E', '9E', '9E', '9E', '9D', '9E', '9E', '9E',
'9E', '9E', 'C0', 'C1', 'C0', 'C1']
Compressed Data (String):
616162626262C0C165656565656565656A6B72747474C2C2C292959B9E9
E9E9E9E9E9D9E9E9E9EC0C1C0C1
Compressed Data (String) sent to client successfully

Key: C0,636485,C1,6F2A6F,C2,8385868788
Key sent to client successfully

-----
SERVER CLOSED
-----
```

Save your Jupyter Notebook and Python files for Task 1.